발표 대본

1-4장 업무 문제와 실행 결과 0:00-2:00

01 전력기기 기술사양 검토 AI

0:00-0:10 | 서비스 제목을 소개합니다. 이번 검증 대상은 변압기입니다.

전력기기 사양 변경을 원문과 대조하는 AI입니다. 현재는 변압기 검토를 구현하고 검증했습니다.

먼저 사용하는 시점입니다.

02 점검 시점은 기술 인계 직전

0:10-0:35 | 수주에서 출고까지의 흐름 중 기술 인계 위치를 짚습니다.

제안하는 사용 시점은 수주 확정 후, 상세 설계와 구매로 넘기기 전입니다. 담당자가 고객 사양과 변경 합의, 내부 인계 내용을 함께 검토하는 시점입니다. 서비스는 이때 문서의 차이와 근거를 먼저 정리해 주는 역할입니다.

변경사항 하나가 빠진 상황을 예로 들어보겠습니다.

03 승인 변경이 인계표에 빠진 경우

0:35-1:10 | 최초 사양, 승인 변경, 내부 인계의 두 값을 순서대로 짚습니다.

이것은 직접 만든 가상 사례입니다. 최초 고객 사양은 결선그룹 Dyn1, 냉각방식 ONAF였습니다. 승인 변경서에는 Dyn11과 OFAF로 바뀌었지만, 내부 인계표에는 처음 값이 남아 있습니다. 최초 사양서와 인계표만 비교하면 놓칠 수 있는 차이입니다. 그래서 담당자가 적용할 변경을 선택한 뒤 비교하도록 했습니다.

이 세 문서를 실제 서비스에 넣은 결과입니다.

04 실제 실행에서 두 항목 차이를 확인

1:10-2:00 | 8개 일치와 2개 불일치를 짚고 실제 결과표의 두 행을 보여줍니다. 양쪽 원문은 부록 10장에 있습니다.

OpenAI의 gpt-4o-mini를 실제 호출한 저장 결과입니다. 열 개 항목 중 여덟 개가 일치했고, 결선그룹과 냉각방식은 불일치였습니다. 미확정 항목은 없었습니다. 담당자는 변경서 1쪽과 인계표 2쪽의 원문을 확인하고 보고서로 남길 수 있습니다. 현재 결과는 가상 변압기 문서의 검증이며, 현업 적용 전에는 독립된 실제 문서 평가가 필요합니다. 핵심은 설계 적합성을 승인하는 것이 아니라, 판단할 근거와 보류 사유를 남기는 것입니다.

이제 그 결과를 만드는 핵심 코드를 보겠습니다.

시간은 5분 발표를 위한 배분안입니다. 실제 발화 시간은 연습으로 조정합니다.

회색은 화면 안내입니다. 본문과 초록색 연결 문장을 이어서 읽습니다.

코드 설명 대본

5-7장 호출 제어와 검증 및 원문 검증 2:00-3:55

05 Agent 실행 흐름과 호출 제어

2:00-2:45 | 검색 계획, 코드 검색, 값 추출, 검증의 호출 위치를 짚은 뒤 call 함수의 제한 조건을 설명합니다.

실행 순서는 Python 코드가 제어합니다. 첫 AI 호출은 검색 계획을 만들고, 검색 함수가 원문을 찾습니다. 두 번째 호출은 검색 결과에서 값을 제안하고, 검증 함수가 이를 확인합니다. 여기서 call 함수는 전체 90초 중 남은 시간과 호출 횟수를 검사합니다. 두 번을 초과하거나 시간이 끝나면 중단하므로 무한 재시도를 하지 않습니다. LangGraph나 MCP는 사용하지 않았고, 이처럼 범위가 정해진 흐름으로 구현했습니다.

그중 원문 검색이 어떻게 이루어지는지 보겠습니다.

06 RAG 검색과 근거 전달

2:45-3:20 | 검색 점수와 상위 두 개를 고르는 코드, 이어서 document_id와 page를 포함한 반환값을 짚습니다.

검색은 벡터 방식이 아닌 키워드와 별칭 기반 RAG입니다. 항목별 원문 후보에 검색어가 포함됐는지 점수를 매기고, 정렬한 뒤 상위 두 개를 선택합니다. 반환할 때는 문서 ID, 사양 항목, 페이지와 원문을 함께 담습니다. 이 결과를 두 번째 AI 호출에 전달하므로, 제안한 값이 어느 문서의 어느 페이지에서 나왔는지 연결할 수 있습니다.

다만 출처가 붙어 있다고 바로 믿지는 않습니다.

07 원문 검증과 미확정 처리

3:20-3:55 | 인용문이 원문에 있는지 확인하는 조건을 짚고, 실패 시 _uncertain으로 넘기는 분기를 설명합니다.

검증 함수는 인용문이 해당 PDF 페이지에 실제 있는지, 검색 결과에도 같은 문서와 항목으로 반환됐는지 확인합니다. 뒤이어 값과 단위가 그 근거에 연결되는지도 검사합니다. 검증에 실패하면 아래 분기에서 값을 확정하지 않고 미확정 상태와 사유를 남깁니다. AI가 그럴듯하게 답했어도 근거를 확인할 수 없다면 비교에 그대로 쓰지 않는 구조입니다.

검증을 통과한 값은 다음 비교 로직으로 넘어갑니다.

코드 설명과 마무리

8-9장 비교 분기와 반복 검사 3:55-5:00

08 사양 비교와 보류 분기

3:55-4:35 | condition 비교, comparable 비교, NEEDS_REVIEW 예외 처리 순서로 실제 코드를 짚습니다.

비교 결과는 AI가 아니라 코드가 결정합니다. 선택한 변경사항을 반영하고 단위를 정규화한 뒤, 먼저 양쪽의 부품이나 운전조건이 같은지 검사합니다. 조건이 다르면 예외를 발생시켜 확인 필요로 보류합니다. 같은 조건일 때만 비교값이 같으면 일치, 다르면 불일치로 분류합니다. 이번 사례의 Dyn11과 Dyn1, OFAF와 ONAF는 이 불일치 분기로 처리됐습니다. 단순히 문자열이 다르다는 이유만으로 곧바로 결론을 내리지는 않습니다.

마지막으로 이 분기들을 어떻게 반복 확인했는지 보겠습니다.

09 의존성 주입과 반복 검사

4:35-5:00 | request_fn에 고정 응답을 넣는 코드와 두 번 호출 및 원본 유지 검사를 짚고 마무리합니다.

테스트에서는 request_fn에 고정 응답을 주입합니다. 실제 API 없이 같은 응답으로 검증을 반복합니다. 여기서는 두 번 호출했는지, 입력 원본을 바꾸지 않았는지 확인합니다. 모델 정확도가 아닌 코드 동작 검사입니다. AI 제안은 코드로 검증하고, 최종 확인은 담당자에게 남깁니다.

부록 활용

10장 원문 근거: 두 불일치 항목의 양쪽 원문을 요청받으면 표시합니다. 변경서 1쪽과 인계표 2쪽을 대조합니다.

11장 실행 확인: 호출 횟수, 추출 제안 수와 최종 비교 항목 수가 다른 이유를 질문받으면 표시합니다. 실행 결과를 정확도 점수로 설명하지 않습니다.

예상 질문과 답변

업무 적용과 구현 범위를 설명할 때 참고합니다.

01 왜 기술 인계 직전에 사용하나요?

고객 사양과 변경 합의가 내부 설계·구매 정보로 넘어가는 시점이기 때문입니다. 세 문서의 차이를 함께 검토하는 업무를 지원하도록 설계했습니다. 이 시점이 모든 회사에서 최적이라고 실증한 것은 아니며, 실제 적용 시에는 회사의 승인·인계 절차에 맞춰야 합니다.

02 불일치라면 제품 설계가 잘못됐다는 뜻인가요?

아닙니다. 선택한 기준 문서와 인계표의 명시 값이 다르다는 뜻입니다. 결선그룹이나 냉각방식의 적합성, 허용 용량, 안전성까지 판정하지는 않습니다. 담당자가 원문과 적용 조건을 확인해 문서 수정 또는 추가 검토 여부를 결정해야 합니다.

03 AI Agent와 RAG를 어떤 방식으로 구현했나요?

Python이 AI의 검색 계획, 코드의 키워드 검색, AI의 값 추출, 코드의 근거 검증을 순서대로 실행합니다. 검색된 원문을 AI 입력으로 전달하는 키워드 기반 RAG이며 벡터 검색은 아닙니다. LangGraph, MCP, 자율적인 멀티 에이전트 협업은 사용하지 않았습니다.

04 왜 호출을 두 번으로 제한했나요?

첫 호출은 문서·항목별 검색어 선택, 두 번째는 실제 검색 결과의 값 추출로 역할을 나눴기 때문입니다. 두 호출은 총 90초 예산을 공유하고, 두 번째 호출은 남은 시간만 사용합니다. 검색 근거가 없으면 두 번째 호출을 생략합니다. 실패 시 자동 재시도나 다른 제공자로의 자동 전환은 하지 않습니다.

예상 질문과 답변

업무 적용과 구현 범위를 설명할 때 참고합니다.

05 원문 인용이 일치하면 AI 답변은 모두 정확한가요?

그렇지는 않습니다. 인용이 실제 원문과 검색 결과에 있는지 확인하는 것은 검증의 일부입니다. 항목명과 값의 연결, 문자 그대로의 값과 단위, 적용 조건과 모호성도 검사합니다. 이 검증을 통과해도 원문 자체의 오류나 지원 범위 밖 설계 조건을 해결하지는 못합니다. 최종 검토는 사람이 맡습니다.

06 일치 8개와 불일치 2개가 정확도인가요?

아닙니다. 직접 만든 가상 문서를 실제 OpenAI API로 처리한 한 사례의 비교 결과입니다. 열 개 최종 항목 중 두 차이를 확인한 결과이지, 현업 문서의 통계적 정확도가 아닙니다. 모델 정확도와 검토 시간 절감은 별도의 독립 문서 평가와 업무 비교 측정이 필요합니다.

07 의존성 주입 테스트와 실제 API 검증은 어떻게 다른가요?

의존성 주입은 request_fn을 고정 응답으로 바꿔 호출 횟수, 원문 검증, 모호성 처리와 실패 분기를 같은 조건에서 반복 검사합니다. 네트워크나 모델 응답의 변동에 의존하지 않는 코드 검사입니다. 실제 API 검증은 선택한 모델과 연결해 전체 흐름을 확인하는 별도 검증이며, 둘 중 하나가 다른 하나를 대체하지 않습니다.

08 다른 전력기기나 실제 회사 문서에도 바로 사용할 수 있나요?

현재 구현·검증 대상은 지정 형식의 변압기 텍스트 PDF와 열 개 사양 항목입니다. 다른 기기에는 해당 항목과 단위, 비교 조건, 문서 형식별 검증을 추가해야 합니다. 실제 적용 전에는 독립 문서 평가, 승인 이력과 접근권한 연결, 외부 API로 보낼 수 있는 정보 범위와 회사 보안정책 확인이 필요합니다.